

NeuraCaR: A Neural Expert for Cache Replacement

CS 486 Project Proposal

Leo Zhu

Tengyi Xu

Problem

Cache replacement decides which object to evict on a miss, directly affecting hit ratio and system performance. Classic heuristics like LRU and LFU are static: they don't adapt patterns. LeCaR [1] improves on this with online learning, deciding for each eviction whether to trust LRU or LFU more based on which has recently made better decisions. But LeCaR only chooses between these two fixed heuristics, so it cannot learn more complex, nonlinear patterns (e.g., combinations of an object's size, recency, and frequency). We propose adding a small trained neural network as a third option, so the system can learn patterns the heuristics miss.

Background

LeCaR picks an eviction action using only the reward observed so far (hit or miss). It maintains an LRU and an LFU “expert,” learning a trust distribution via a multiplicative-weights update favoring whichever avoided recent misses, a case of the general *tracking-the-best-expert* framework (which holds for any number of experts) [2].

`libCacheSim` [3], our simulator, already has two ML-based baselines: LHD [4] (statistical, not a neural network) and LRB [5] (gradient-boosted tree, replaces heuristics). Neither is a neural network, and neither falls back on cheap heuristics, which is where the NN expert fits in.

Motivation

Can a small neural network, used as one option alongside LRU and LFU, learn patterns the heuristics miss, while still falling back on those heuristics if it performs poorly on an unfamiliar workload? This lets us apply both supervised learning (to train the network) and RL-style updates (to choose between options) in one project.

Proposed Methods

- **Neural network:** a small feedforward network with 2 hidden layers, taking each object's recency, frequency, time since last access, and size as input, predicting the probability the object will be reused before it is evicted. Labels come from checking whether each object is accessed again within a fixed time window (standard supervised binary classification, trained with backpropagation). We train on the first part of each trace and test on the later part.
- **Combining with LRU/LFU:** extend LeCaR's weight-update procedure from 2 options to 3 (LRU, LFU, NN); at each eviction the system picks one option by weight, observes the reward, and updates all three weights.
- **Baselines, traces, and evaluation:** compare against `libCacheSim`'s built-in LRU, LFU, ARC, LeCaR (original version), LHD, and LRB on 2–3 real traces, measuring hit ratio across cache sizes, a feature ablation, and performance on a workload that changes partway through, to see whether our method falls back to LRU/LFU when the network is wrong, which LRB cannot do.

References

- [1] G. Vietri, L. V. Rodriguez, W. A. Martinez, S. Lyons, J. Liu, R. Rangaswami, M. Zhao, and G. Narasimhan. Driving Cache Replacement with ML-based LeCaR. *USENIX HotStorage*, 2018.
- [2] M. Herbster and M. K. Warmuth. Tracking the Best Expert. *Machine Learning*, 32(2):151–178, 1998.
- [3] J. Yang et al. `libCacheSim`: A High-Performance Cache Simulator. <https://github.com/cacheMon/libCacheSim>.
- [4] N. Beckmann, H. Chen, and A. Sanchez. LHD: Improving Cache Hit Rate by Maximizing Hit Density. *USENIX NSDI*, 2018.
- [5] Z. Song, D. S. Berger, K. Li, and W. Lloyd. Learning Relaxed Belady for Content Distribution Network Caching. *USENIX NSDI*, 2020.